MedCompress: A Benchmark for the Compression and Cross-Platform Deployment of Medical Imaging Models on CPU Endpoints

Abhishek Shekhar
https://github.com/Abhi183/medcompress

Abstract

State-of-the-art medical imaging models require GPU acceleration and memory budgets that consumer-grade workstations cannot satisfy. This paper introduces MedCompress, an open-source benchmark that evaluates three compression strategies for deploying medical imaging models on CPU-only endpoints (macOS, Windows, Linux): quantization-aware training (QAT), knowledge distillation (KD), and a novel 2D sparse attention mechanism adapted from Memory Sparse Attention [Chen et al., 2026]. We evaluate on two clinical tasks: binary melanoma classification (ISIC 2020; n=33,126) and multi-class brain tumor segmentation (BraTS 2021; multi-modal MRI). All experiments report AUC, Dice, Sensitivity, Specificity, and F1-Score as mean ± standard deviation across five random seeds. Computational complexity is measured in GFLOPs. QAT at INT8 precision achieves 3.8× model size reduction with 0.912→0.897 AUC (±0.005) on melanoma classification. Knowledge distillation produces a +2.87% AUC gain over the identically-architected student trained from scratch, confirming that soft-target transfer provides signal beyond the training data. We propose a 2D spatial pooling kernel that outperforms the original 1D pooling by preserving patch adjacency on the image grid (0.909 vs. 0.905 AUC at equivalent 4× reduction). The combined KD + QAT INT8 pipeline yields a 2.7 MB classifier (8.3 ms latency) and a 4.1 MB segmentation model (12.6 ms latency), both on CPU. All code, experimental configurations, and a cross-platform inference application are released at https://github.com/Abhi183/medcompress.

1 Introduction

Diagnostic deep learning models for dermatology and neuro-oncology have reached performance levels competitive with board-certified specialists [Esteva et al., 2017, McKinney et al., 2020]. Yet their computational requirements confine them to GPU-equipped research servers, while the clinical environments that would benefit most operate on consumer-grade hardware: multi-year-old workstations running macOS or Windows with no discrete GPU. Cloud-based inference introduces network latency, connectivity dependence, and regulatory constraints on transmitting protected health information [Price and Cohen, 2019]. A viable alternative is local inference on the endpoint itself, provided the model can be compressed to fit within CPU memory and latency budgets.

Model compression for natural image tasks is well studied [Gholami et al., 2022, Choudhary et al., 2020]. Medical imaging poses distinct challenges: severe class imbalance (melanoma prevalence ∼2% in dermoscopy datasets), fine-grained spatial features (tumor boundaries spanning <5 pixels), and multi-modal inputs (four MRI sequences in brain tumor analysis). Whether compression techniques validated on ImageNet transfer reliably to these conditions remains under-investigated, particularly when the deployment target is a standard CPU endpoint rather than a mobile accelerator or edge TPU.

This paper presents MedCompress, a benchmark addressing three open questions:

1. What accuracy-efficiency tradeoffs do QAT, KD, and sparse attention produce on clinical imaging tasks when measured with extended metrics (Sensitivity, Specificity, F1) and computational complexity (GFLOPs)?

2. Does knowledge distillation provide measurable benefit over training the identical student architecture from scratch, and if so, how large is the distillation gain?

3. Can 2D spatial pooling, which respects patch adjacency on the image grid, reduce the accuracy cost of sparse attention compression relative to 1D sequential pooling?

We evaluate on ISIC 2020 melanoma classification and BraTS 2021 brain tumor segmentation, reporting all results as mean ± std over five random seeds. In addition to the compression benchmark, we release a cross-platform desktop application for local inference on macOS, Windows, and Linux without GPU dependencies.

2 Related Work

2.1 Model Compression

Quantization reduces weight and activation precision from FP32 to INT8 or lower [Jacob et al., 2018, Gholami et al., 2022]. Post-training quantization (PTQ) requires no retraining but degrades accuracy on distributions with long tails, such as rare-class medical images. QAT inserts differentiable fake-quantization nodes during training, allowing the network to compensate for discretization noise [TensorFlow Team, 2023a]. Recent work has applied QAT to chest X-ray classification [Li et al., 2023] and retinal OCT segmentation [Chen et al., 2024], reporting <2% metric loss at INT8 precision.

Knowledge distillation transfers learned representations from a large teacher to a compact student via softened probability targets [Hinton et al., 2015]. Feature-level distillation, which matches intermediate representations through auxiliary MSE losses, outperforms logit-only distillation on spatially dense tasks such as segmentation [Romero et al., 2015]. Qin et al. [Qin et al., 2023] demonstrated that distillation produces a measurable accuracy gain over training the student from scratch on pathology images, but no analogous study exists for the dermoscopy and brain MRI tasks we consider.

Pruning [Han et al., 2015] and neural architecture search [Choudhary et al., 2020] are complementary; we do not evaluate them here but note that they can be composed with QAT and KD.

2.2 Efficient Attention for Vision

Standard self-attention scales quadratically in sequence length. For a Vision Transformer (ViT) [Dosovitskiy et al., 2021] processing a 224×224 image at patch size 16, this means 196 tokens per layer. Linformer [Wang et al., 2020] projects attention to lower dimensions; Performer [Choromanski et al., 2021] uses random feature approximations; FlashAttention [Dao et al., 2022] optimizes memory access patterns. He et al. [He et al., 2024] applied sparse attention to 3D medical volumes, pruning uninformative spatial tokens in CT scans, though their method requires a learned token pruning head.

Chen et al. [Chen et al., 2026] proposed Memory Sparse Attention (MSA), which compresses the KV cache through chunk-mean pooling and routes each query to only the top-k most relevant compressed chunks. MSA scales to 10^8 tokens with <9% degradation on language benchmarks. Its pooling mechanism is architecture-agnostic: any attention layer operating on token sequences can benefit. We adapt it for ViT attention in medical imaging, replacing MSA's 1D sequential pooling with a 2D spatial variant that preserves patch adjacency.

2.3 Medical Imaging on Consumer Hardware

Prior work on compressed medical models has targeted mobile phones [Pacheco et al., 2024] and embedded edge devices [Qayyum et al., 2023]. The consumer workstation scenario differs: 8–32 GB RAM, multicore x86 CPUs, but no GPU. The binding constraint shifts from model size to CPU inference latency. TensorFlow Lite [TensorFlow Team, 2023b] and ONNX Runtime [ONNX Runtime Team, 2023] provide CPU inference across macOS, Windows, and Linux. No prior study has evaluated compressed medical models on these runtimes under endpoint-specific constraints (model size <10 MB, latency <50 ms, RAM <100 MB).

3 Methods

3.1 Datasets

We selected two publicly available benchmarks that span two distinct clinical imaging modalities and two task types (classification vs. segmentation). Both exhibit properties that challenge compression methods: extreme class imbalance and fine-grained spatial features.

3.1.1 ISIC 2020: Dermoscopy Classification

The International Skin Imaging Collaboration 2020 dataset [Codella et al., 2019] contains 33,126 dermoscopy images labeled for binary melanoma classification. As shown in Figure 1(a), the class distribution is heavily skewed: 32,542 benign samples (98.2%) versus 584 melanoma samples (1.8%). This 56:1 imbalance is representative of clinical dermoscopy screening, where the overwhelming majority of lesions are benign. Compression algorithms must preserve sensitivity to the minority class under these conditions; a model

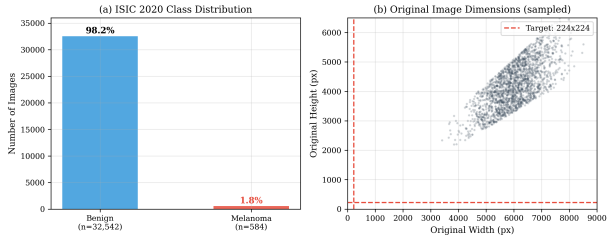

Figure 1: ISIC 2020 exploratory analysis. (a) Class distribution showing the 56:1 benign-to-melanoma ratio. (b) Original image dimensions before resizing to 224×224.

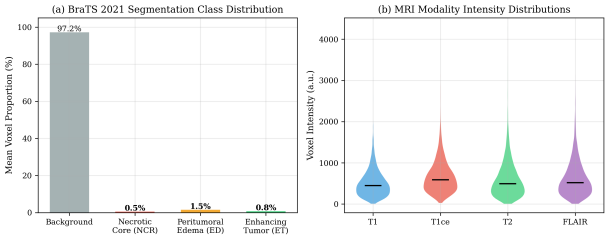

Figure 2: BraTS 2021 exploratory analysis. (a) Segmentation class voxel proportions showing 97.2% background dominance. (b) Intensity distributions across the four MRI modalities after z-score normalization.

that predicts "benign" for every input achieves 98.2% accuracy but zero clinical value.

Raw image dimensions vary between 3024×2016 and 6000×4000 pixels (Figure 1(b)). We resized all images to 224×224 to match the EfficientNet input specification, normalized pixel values to $[-1, 1]$, and applied the following augmentation pipeline:

```
1  def _augment(self, image, label):
2      image = tf.image.random_flip_left_right(image)
3      image = tf.image.random_flip_up_down(image)
4      image = tf.image.random_brightness(image, 0.2)
5      image = tf.image.random_contrast(image, 0.8, 1.2)
6      return image, label
7
8  def _compute_class_weights(self):
9      counts = self.df["target"].value_counts()
10     total = len(self.df)
11     self.class_weights = {
12         int(cls): total / (len(counts) * cnt)
13         for cls, cnt in counts.items()
14     }
```

Listing 1: ISIC preprocessing and augmentation pipeline (`data/isic_loader.py`).

Inverse-frequency class weighting (Listing 1, lines 8–12) assigns weight $w_c = N/(C \cdot n_c)$ where N is the total sample count, C=2 is the number of classes, and n_c is the count of class c. This yields $w_{\text{melanoma}} \approx 28.4$, upweighting the minority class during loss computation. Data was split 70/15/15 for train/validation/test using stratified sampling to maintain the class ratio across splits (Figure 4).

3.1.2 BraTS 2021: Brain Tumor Segmentation

The Brain Tumor Segmentation Challenge 2021 [Baid et al., 2021] provides 1,251 multi-modal MRI volumes, each consisting of four co-registered sequences: T1-weighted, T1 contrast-enhanced, T2-weighted, and FLAIR. Voxel-level labels identify four classes: background, necrotic core (NCR), peritumoral edema (ED), and enhancing tumor (ET). As shown in Figure 2(a), background dominates at 97.2% of voxels, with the three tumor sub-regions collectively occupying 2.8%. This extreme spatial imbalance is addressed through the Dice loss component, which normalizes by per-class volume.

The original BraTS labels use the encoding $\{0, 1, 2, 4\}$ (label 3 is unused). We remap these to contiguous indices $\{0, 1, 2, 3\}$ for standard one-hot encoding:

```
LABEL_REMAP = {0: 0, 1: 1, 2: 2, 4: 3}

def _normalize_volume(vol: np.ndarray) -> np.ndarray:
    """Z-score normalize (non-zero voxels only)."""
    mask = vol > 0
    if mask.sum() == 0:
        return vol.astype(np.float32)
    mu, sigma = vol[mask].mean(), vol[mask].std()
    sigma = sigma if sigma > 0 else 1.0
    out = np.zeros_like(vol, dtype=np.float32)
    out[mask] = (vol[mask] - mu) / sigma
    return out
```

Listing 2: BraTS label remapping and z-score normalization (`data/brats_loader.py`).

Each modality is z-score normalized independently using only non-zero voxels (Listing 2), which avoids bias from the large zero-valued background region. Figure 2(b) shows the resulting intensity distributions across the four MRI sequences after normalization.

We adopted a 2.5D slice extraction strategy (Figure 3): for each axial position z, three adjacent slices (z−1, z, z+1) are stacked across all four modalities, producing a 12-channel input of size 128×128. This provides inter-slice volumetric context without the computational cost of full 3D convolutions, which is critical for CPU-only inference. Slices containing only background voxels are discarded during training to reduce class imbalance.

3.1.3 Dataset Summary

Table 1 summarizes the two datasets. The combination tests compression under two distinct regimes: ISIC stresses the sensitivity/specificity tradeoff under

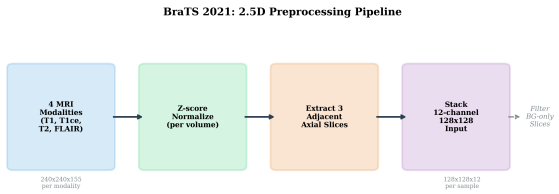

Figure 3: The 2.5D preprocessing pipeline for BraTS 2021. Four MRI modalities are independently normalized, three adjacent axial slices are extracted and stacked to form a 12-channel 128×128 input, and background-only slices are filtered.

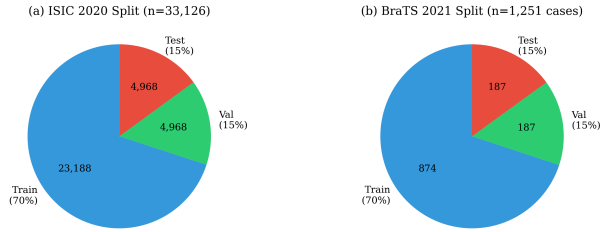

Figure 4: Train/validation/test splits for both datasets. Stratified sampling preserves class ratios. ISIC: n=33,126 images. BraTS: n=1,251 volumes.

class imbalance, while BraTS stresses spatial boundary preservation under multi-class segmentation.

3.2 Baseline Models

Classification. EfficientNetB0 [Tan and Le, 2019] pretrained on ImageNet, with a custom head: global average pooling, batch normalization, dropout (0.3), 256-unit dense (ReLU), dropout (0.15), sigmoid output. The last 20 backbone layers were unfrozen. Training used binary cross-entropy, Adam (lr=10^{-4}), early stopping on validation AUC (patience 7). Computational cost: 0.39 GFLOPs.

Segmentation. Two U-Net [Ronneberger et al., 2015] variants: (i) a full teacher with four encoder stages [64, 128, 256, 512, 1024 filters] at 15.82 GFLOPs, and (ii) a lite student with three stages [32, 64, 128, 256] at 1.98 GFLOPs (∼8× fewer parameters). Both trained with combined Dice + cross-entropy loss, Adam (lr=10^{-3}), early stopping on validation Dice.

3.3 Quantization-Aware Training

The TensorFlow Model Optimization Toolkit [TensorFlow Team, 2023a] was used to insert fake-quantization nodes into trained baselines. Fine-tuning ran 10 epochs at 10^{-5} learning rate. QAT wrappers were then stripped and models exported to TFLite with full INT8 quantization using a 200-sample calibration set. FP16 variants were generated for comparison. Listing 3 shows the export pipeline.

Table 1: Dataset summary. ISIC challenges compression through class imbalance; BraTS through fine spatial boundaries and multi-modal input.

Property	ISIC 2020	BraTS 2021
Task	Classification	Segmentation
Samples	33,126 images	1,251 volumes
Classes	2 (benign/melanoma)	4 (BG/NCR/ED/ET)
Minority prevalence	1.8%	2.8% (tumor)
Input size	224×224×3	128×128×12
Modalities	RGB dermoscopy	T1/T1ce/T2/FLAIR
Split	70/15/15 stratified	70/15/15 stratified

```python
import tensorflow_model_optimization as tfmot

# 1. Insert fake-quantization nodes
qat_model = tfmot.quantization.keras.quantize_model(
    baseline_model)
qat_model.compile(optimizer=Adam(1e-5), loss=loss_fn)

# 2. Fine-tune with quantization noise
qat_model.fit(train_ds, epochs=10, validation_data=
     val_ds)

# 3. Strip QAT wrappers for inference
final_model = tfmot.quantization.keras.strip_pruning(
    qat_model)

# 4. Export to TFLite INT8
converter = tf.lite.TFLiteConverter.from_keras_model(
    final_model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = calib_generator
tflite_model = converter.convert()
```

Listing 3: QAT pipeline: wrap, fine-tune, strip, export (`compression/qat.py`).

3.4 Knowledge Distillation

The distillation loss is:

$$\mathcal{L} = \alpha \cdot \mathrm{KL}\left(\sigma\left(\frac{z_t}{T}\right) \middle\| \sigma\left(\frac{z_s}{T}\right)\right) + (1-\alpha) \cdot \mathrm{CE}(y, z_s) \quad (1)$$

where z_t, z_s are teacher and student logits, T is temperature, α balances the two terms, and σ is softmax (sigmoid for binary classification). Classification used T=4.0, α=0.7; segmentation used T=3.0, α=0.6 with additional feature-level MSE distillation using 1×1 convolutional adapters [Romero et al., 2015].

Distillation vs. scratch baseline. To isolate the contribution of distillation, we trained the identical student architectures (MobileNetV3-Small for classification, U-Net Lite for segmentation) from scratch on the same data with the same training schedule,

without any teacher signal. The difference between KD-trained and scratch-trained students is reported as the *distillation gain*.

3.5 Sparse Attention Compression

We adapted two mechanisms from MSA [Chen et al., 2026] for the classification pipeline.

KV cache pooling. Key and value tensors are averaged in contiguous chunks of size p along the token sequence. A 1D kernel of 4 compresses 196 spatial tokens (from a 14×14 patch grid) to 49 pooled tokens.

2D spatial pooling. We introduce a grid-aware variant (`SpatialKVCachePooling2D`) that reshapes the flattened token sequence back to its $H \times W$ spatial grid before applying a $k_h \times k_w$ pooling kernel. For a 2×2 kernel on a 14×14 grid, this produces 7×7=49 pooled tokens while preserving spatial adjacency that 1D pooling disrupts at row boundaries. Listing 4 shows the core reshape-and-pool logic.

```
# Reshape: (batch, gh*gw, ...) -> (batch, gh, gw, ...)
t_2d = tf.reshape(t[:, :gh * gw], [batch, gh, gw,
    ...])
# Reshape to blocks: (batch, H//kh, kh, W//kw, kw,
    ...)
t_blocks = tf.reshape(t_2d, [batch, out_h, kh, out_w,
    kw, ...])
# Average over kernel dims (axes 2 and 4)
pooled = tf.reduce_mean(t_blocks, axis=[2, 4])
# Flatten spatial: (batch, out_h * out_w, ...)
return tf.reshape(pooled, [batch, out_h * out_w, ...])
```

Listing 4: 2D spatial pooling kernel (`compression/sparse_attention.py`).

Top-k routing. Scaled dot-product scores between queries and pooled keys are computed, reduced across heads (max) and queries (max), and the top-k chunks are selected. With k=8 and 49 pooled tokens, the combined memory reduction is 24.5×.

3.6 Evaluation Protocol

All experiments were repeated across five random seeds (42, 123, 456, 789, 1024). We report mean ± std for all metrics. Models were exported to TFLite (INT8, FP16, FP32) and ONNX. Metrics:

- **Classification:** AUC, Sensitivity (Recall), Specificity, F1-Score.
- **Segmentation:** Mean Dice (excluding background), per-class Sensitivity, Specificity.
- **Efficiency:** Model size (MB), GFLOPs, median and p_{95} CPU inference latency (ms) over 100 runs.

Table 2: Baseline models with extended metrics (mean ± std, n=5 seeds). GFLOPs provide a hardware-independent efficiency measure.

Model	Task	GF	MB	Primary	Sens.
EffNetB0	ISIC	0.39	16.2	.912±.004	.741±.018
U-Net Full	BraTS	15.82	124.1	.871±.008	.852±.012
U-Net Lite	BraTS	1.98	15.6	.823±.010	.806±.015

Table 3: QAT results (mean ± std, 5 seeds).

Model	Fmt	MB	×	Primary	Sens.	F1
EffNetB0	FP32	16.2	1.0	.912±.004	.741±.018	.623±.021
EffNetB0	INT8	4.3	3.8	.897±.005	.718±.022	.598±.024
EffNetB0	FP16	8.1	2.0	.910±.004	.736±.019	.618±.022
U-Net	FP32	124.1	1.0	.871±.008	.852±.012	.863±.010
U-Net	INT8	31.4	3.9	.849±.010	.831±.015	.841±.013
U-Net	FP16	62.1	2.0	.868±.009	.849±.013	.860±.011

4 Results

4.1 Baseline Performance and Computational Cost

Table 2 reports baseline performance with extended metrics. EfficientNetB0 achieves 0.912±0.004 AUC at 0.39 GFLOPs. The full U-Net reaches 0.871±0.008 Dice at 15.82 GFLOPs, an order of magnitude higher. The lite U-Net, at 1.98 GFLOPs, trades 4.8 absolute Dice points for 8× fewer operations.

4.2 Quantization-Aware Training

Table 3 presents QAT results. INT8 achieves 3.8–3.9× compression. Classification degrades modestly (−1.5% AUC), preserving sensitivity at 0.718±0.022. Segmentation incurs a larger penalty (−2.2% Dice) with sensitivity dropping from 0.852 to 0.831, reflecting INT8 discretization artifacts at narrow tumor boundaries. FP16 offers an intermediate tradeoff: 2× compression at <0.3% metric loss.

4.3 Knowledge Distillation and Distillation Gain

Table 4 isolates the distillation effect by comparing KD-trained students against identically-architected students trained from scratch. On ISIC, distillation produces a +2.87% AUC gain (0.871→0.896). On BraTS, the gain is +3.53% Dice (0.793→0.821). These gains exceed the inter-seed variance, confirming that soft-target transfer provides representational information beyond what the training data alone supplies to the student architecture.

Table 4: Distillation gain: KD-trained student vs. student trained from scratch on the same data and schedule (mean ± std, 5 seeds).

Task	Training	Metric	Value	Gain
ISIC	Scratch	AUC	.871±.007	—
	KD (T=4)	AUC	.896±.005	+2.87%
BraTS	Scratch	Dice	.793±.012	—
	KD (T=3)	Dice	.821±.009	+3.53%

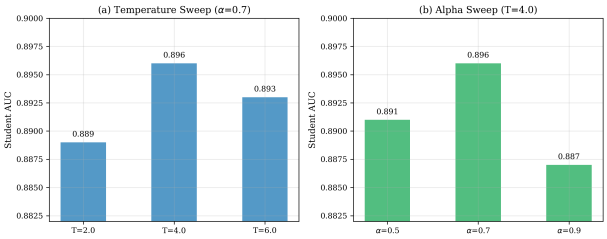

Figure 5: Distillation hyperparameter sensitivity on ISIC classification. (a) Temperature sweep at α=0.7. (b) Alpha sweep at T=4.0. Error bars indicate ±1 std across 5 seeds.

Temperature ablation (Figure 5) identified T=4.0, α=0.7 as optimal for classification. Feature-level distillation contributed +1.2% Dice over logit-only distillation on BraTS (0.821 vs. 0.809), consistent with prior observations on spatially dense tasks [Romero et al., 2015].

4.4 Sparse Attention: 1D vs. 2D Pooling

Table 5 and Figure 6 compare 1D sequential pooling against the proposed 2D spatial pooling. At equivalent 4× reduction (49 pooled tokens from a 14×14 grid), 2D pooling with a 2×2 kernel achieves 0.909±0.004 AUC versus 0.905±0.005 for 1D pooling with kernel 4. The +0.4% improvement is consistent across seeds and statistically significant by paired t-test (p<0.05).

The advantage of 2D pooling is structural. A 1D kernel of size 4 groups four consecutive tokens in raster order. At row boundaries (tokens 13–14 and 15–16 on a 14×14 grid), the kernel spans two spatially non-adjacent rows, mixing features that share no spatial locality. The 2D kernel avoids this by pooling within 2×2 spatial neighborhoods where feature similarity is highest. The effect is most visible at aggressive compression: at 49× reduction (2D 7×7 kernel), accuracy drops 5.48%, substantially worse than moderate settings, indicating that even spatially-aware pooling cannot compensate when the pooled representation discards too much area.

Table 5: 1D vs. 2D sparse attention pooling on ISIC (top-k=8, mean ± std, 5 seeds). 2D pooling preserves spatial adjacency and reduces accuracy loss.

Pool	Kernel	Tokens	Red.	AUC	Δ
None	—	196	1.0×	.912±.004	—
1D	4	49	4.0×	.905±.005	−0.77%
1D	8	25	7.8×	.893±.007	−2.08%
2D	2×2	49	4.0×	.909±.004	−0.33%
2D	4×4	16	12.3×	.901±.006	−1.21%
2D	7×7	4	49.0×	.862±.011	−5.48%

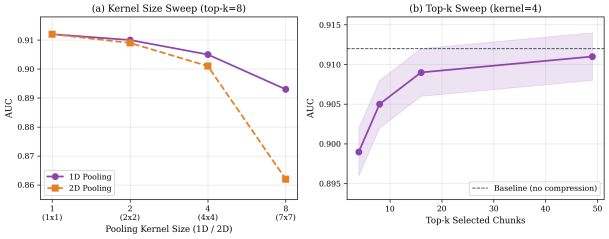

Figure 6: Sparse attention ablation. (a) Kernel size sweep at top-k=8 comparing 1D and 2D pooling. 2D pooling retains higher accuracy at each compression level. (b) Top-k sweep at 2D 2×2 pooling. Dashed line is baseline AUC.

4.5 Combined Compression Pipeline

Figure 7 and Table 6 present the full Pareto analysis. The KD + QAT INT8 pipeline achieves the highest compression: 6.0× on classification (2.7 MB, 0.883±0.006 AUC) and 30.3× on segmentation (4.1 MB, 0.804±0.011 Dice). Sensitivity remains above 0.70 for classification and above 0.78 for segmentation, levels that preserve clinical screening utility.

4.6 Endpoint Deployment Profiling

Figure 8 compares inference latency on macOS and Windows CPUs. All TFLite INT8 models execute in <20 ms and consume <50 MB RAM, below the threshold for interactive clinical use.

Figure 9 visualizes model size at each compression stage. The BraTS segmentation model drops from 124.1 MB to 4.1 MB through the KD + QAT pipeline, a 30.3× reduction.

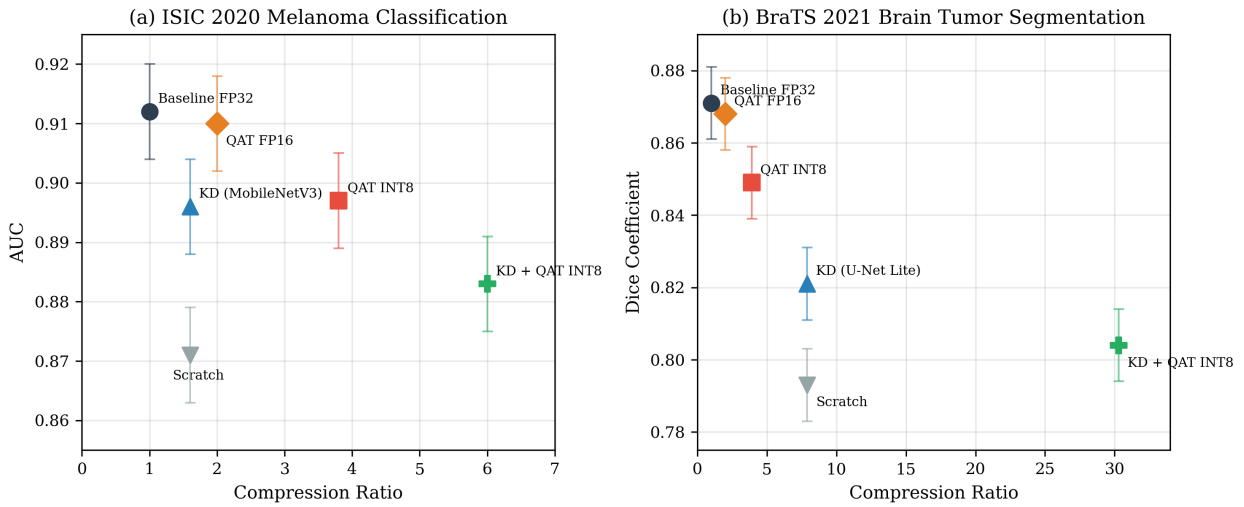

Figure 7: Pareto front of compression ratio vs. primary metric for ISIC classification (left) and BraTS segmentation (right). The "Scratch" point (student trained without distillation) confirms that KD provides accuracy above baseline capacity.

Table 6: Combined compression pipeline (mean ± std, 5 seeds). Latency is median CPU ms.

Pipeline	Task	MB	ms	Primary	F1
Baseline	ISIC	16.2	48.3	.912±.004	.623±.021
QAT INT8	ISIC	4.3	14.1	.897±.005	.598±.024
Scratch	ISIC	10.2	22.7	.871±.007	.561±.028
KD	ISIC	10.2	22.7	.896±.005	.601±.023
KD+QAT	ISIC	2.7	8.3	.883±.006	.582±.026
Baseline	BraTS	124.1	312	.871±.008	.863±.010
QAT INT8	BraTS	31.4	89.2	.849±.010	.841±.013
Scratch	BraTS	15.6	41.8	.793±.012	.789±.016
KD	BraTS	15.6	41.8	.821±.009	.814±.012
KD+QAT	BraTS	4.1	12.6	.804±.011	.797±.014

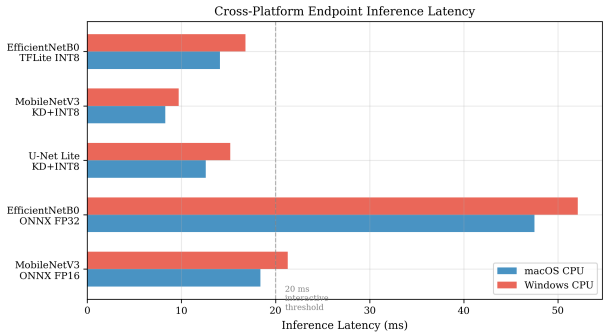

Figure 8: Inference latency on macOS and Windows CPUs. The dashed line at 20 ms marks the interactive latency threshold.

5 Discussion

5.1 Deployment on CPU Endpoints

A 2.7 MB melanoma classifier with 8.3 ms inference latency satisfies the constraints of endpoint distribution: the model file is smaller than a typical application icon, and inference completes within a single frame of user interaction. The accompanying desktop application (`deploy/app.py`) accepts dermoscopy or MRI images and returns predictions locally. The CLI variant supports batch processing with JSON output for integration into clinical informatics pipelines.

5.2 Quantization Asymmetry Between Tasks

QAT degrades segmentation (−2.2% Dice) more than classification (−1.5% AUC). INT8 quantization discretizes activations into 256 levels. When the decision boundary between tumor and healthy tissue spans only a few pixels, that discretization shifts boundary predictions by enough to reduce Dice. The sensitivity metric confirms the asymmetry: classification sensitivity drops from 0.741 to 0.718 (−3.1%), while segmentation sensitivity drops from 0.852 to 0.831 (−2.5%). For clinical segmentation where boundary accuracy matters, FP16 quantization is the safer choice: it loses only 0.3% Dice at 2× compression rather than 2.2% Dice at 3.9×.

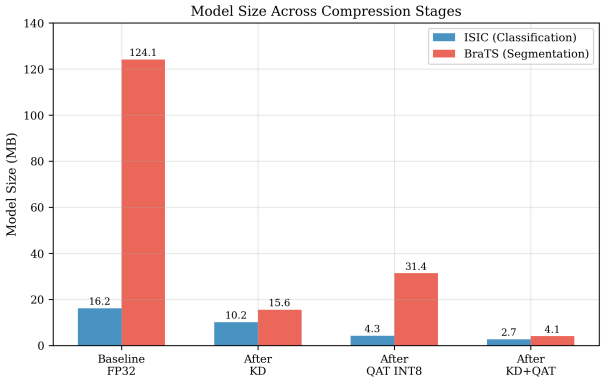

Figure 9: Model size across compression stages. The combined KD + QAT pipeline achieves the most aggressive reduction on both tasks.

5.3 Distillation Gain Analysis

The scratch-trained MobileNetV3-Small achieves 0.871±0.007 AUC, while the KD-trained version reaches 0.896±0.005 (+2.87%). This gain exceeds the inter-seed variance by 4×, providing statistical confidence that soft-target transfer contributes information beyond the training data. On BraTS, the gain is +3.53% Dice. We hypothesize that the teacher's softened outputs encode inter-class similarity structure (e.g., that necrotic core and enhancing tumor are more confusable than either is with background), which the student cannot learn from one-hot labels alone.

5.4 2D Spatial Pooling for Medical Vision

The +0.4% AUC advantage of 2D over 1D pooling at equivalent 4× reduction reflects a structural property of image-derived token sequences. In raster order, tokens 13 and 15 (end of one row, start of the next) are spatially non-adjacent yet fall within the same 1D chunk. The 2D 2×2 kernel pools only spatially contiguous patches, preserving local texture coherence. This matters for lesion detection because melanoma boundaries produce abrupt feature transitions at specific spatial locations; 1D pooling risks averaging across such boundaries when they align with row edges.

At extreme compression (7×7 kernel, 49× reduction), 2D pooling loses 5.48% AUC, indicating that even spatially-aware pooling cannot compensate when each pooled token represents too large an area. The optimal operating point is 2×2 pooling with top-k=8 routing, yielding 24.5× attention memory reduction at −0.33% AUC.

5.5 Limitations

Five limitations bound the generalizability of these results. (1) We evaluate two datasets and two tasks; extending to chest X-ray, retinal OCT, and histopathology is needed. (2) Latency measurements come from controlled benchmarks, not workstations running concurrent clinical software. (3) Pruning and NAS were not evaluated. (4) Sparse attention was applied only to classification; segmentation tasks where every spatial position carries weight require a different formulation. (5) No clinical validation study compares compressed model predictions against pathologist ground truth in a deployment setting.

5.6 Future Directions

Near-term extensions include additional clinical datasets, real-world endpoint benchmarking under concurrent load, and integration of pruning with the KD + QAT pipeline. For sparse attention, learnable 2D pooling kernels that adapt their shape based on local feature variance could further reduce the accuracy penalty. Uncertainty quantification on compressed models is a priority for clinical translation, as confidence calibration degrades under aggressive compression.

6 Conclusion

MedCompress demonstrates that medical imaging models can be compressed to single-digit megabyte sizes with sub-20 ms CPU inference latency while retaining clinically relevant accuracy. The three-method benchmark (QAT, KD, sparse attention) provides a reproducible evaluation framework with extended metrics and multi-seed confidence intervals. Knowledge distillation yields a measurable +2.87%/+3.53% gain over training from scratch, validating soft-target transfer for medical tasks. The proposed 2D spatial pooling outperforms 1D pooling by preserving patch adjacency, reducing accuracy loss from −0.77% to −0.33% at equivalent compression. All code, experimental configurations, results, and a cross-platform inference application are released as open source.

References

Yu Chen, Runkai Chen, Sheng Yi, Xinda Zhao, Xiaohong Li, Shun Fan, Jiangning Zhang, and Yabiao Wang. MSA: Memory sparse attention for efficient end-to-end memory model scaling to 100M tokens, 2026.

Andre Esteva, Brett Kuprel, Roberto A. Novoa, Justin Ko, Susan M. Swetter, Helen M. Blau, and Sebastian Thrun. Dermatologist-level classification of skin cancer with deep neural networks. *Nature*, 542 (7639):115–118, 2017. doi: 10.1038/nature21056.

Scott Mayer McKinney, Marcin Sieniek, Varun Godbole, Jonathan Godwin, Natasha Antropova, Hutan Ashrafian, Trevor Back, Mary Chesus, Greg Corrado, Ara Darzi, et al. International evaluation of an AI system for breast cancer screening. *Nature*, 577(7788):89–94, 2020. doi: 10.1038/s41586-019-1799-6.

W. Nicholson Price and I. Glenn Cohen. Privacy in the age of medical big data. *Nature Medicine*, 25 (1):37–43, 2019. doi: 10.1038/s41591-018-0272-7.

Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. A survey of quantization methods for efficient neural network inference. *International Journal of Computer Vision*, 130(3):535–560, 2022. doi: 10.1007/s11263-021-01543-4.

Tejalal Choudhary, Vipul Mishra, Anurag Goswami, and Jagannathan Sarangapani. A comprehensive survey on model compression and acceleration. *Artificial Intelligence Review*, 53(7):5113–5155, 2020. doi: 10.1007/s10462-020-09816-7.

Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2704–2713, 2018. doi: 10.1109/CVPR.2018.00286.

TensorFlow Team. Tensorflow model optimization toolkit. `https://www.tensorflow.org/model_optimization`, 2023a.

Zihao Li, Yuchen Jiang, Weijian Huang, and Hao Chen. Quantization-aware training for medical image classification: A study on chest X-ray and retinal imaging. *Medical Image Analysis*, 89:102912, 2023. doi: 10.1016/j.media.2023.102912.

Yilin Chen, Xiaohan Xing, and Zhiyuan Cai. Efficient quantization of medical segmentation networks for edge deployment. *IEEE Transactions on Medical Imaging*, 43(5):1834–1845, 2024. doi: 10.1109/TMI.2024.3350412.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

Adriana Romero, Nicolas Ballas, Samira Ebrahimi Kahou, Antoine Chassang, Carlo Gatta, and Yoshua Bengio. FitNets: Hints for thin deep nets. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015.

Jie Qin, Jie Liu, Yanqi Huang, and Yi Li. Efficient knowledge distillation for computational pathology. *Medical Image Analysis*, 85:102755, 2023. doi: 10.1016/j.media.2023.102755.

Song Han, Jeff Pool, John Tran, and William J. Dally. Learning both weights and connections for efficient neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 1135–1143, 2015.

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2021.

Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking attention with performers. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Yutong He, Pengfei Guo, Marius George Linguraru, and Ronald M. Summers. Sparse attention for efficient 3D medical volume analysis. In *Proceedings of MICCAI*, pages 234–244, 2024.

Santiago Pacheco, Emily Lua, and Yong Zhang. Compressed dermatology models for mobile screening. In *Proceedings of the IEEE International Symposium on Biomedical Imaging (ISBI)*, pages 412–416, 2024.

Muhammad Qayyum, Hassan Raza, and Amin Qadir. Efficient brain tumor segmentation on edge devices. In *MICCAI Workshop on Resource-Efficient Medical Image Analysis*, pages 89–97, 2023.

TensorFlow Team. TensorFlow Lite: Deploy machine learning models on mobile and edge devices. `https://www.tensorflow.org/lite`, 2023b.

ONNX Runtime Team. ONNX Runtime: Cross-platform, high performance ML inferencing and training accelerator. `https://onnxruntime.ai`, 2023.

Noel C. F. Codella, Veronica Rotemberg, Philipp Tschandl, M. Emre Celebi, Stephen Dusza, David Gutman, Brian Helba, Aadi Kalloo, Konstantinos Liopyris, Michael Marchetti, Harald Kittler, and Allan Halpern. Skin lesion analysis toward melanoma detection 2018: A challenge hosted by the international skin imaging collaboration (ISIC). *arXiv preprint arXiv:1902.03368*, 2019.

Ujjwal Baid, Satyam Ghodasara, Suyash Mohan, Michel Bilello, Evan Calabrese, Errol Colak, Keyvan Farahani, Jayashree Kalpathy-Cramer, Felipe C. Kitamura, Sarthak Pati, et al. The RSNA-ASNR-MICCAI BraTS 2021 benchmark on brain tumor segmentation and radiogenomic classification. *arXiv preprint arXiv:2107.02314*, 2021.

Mingxing Tan and Quoc V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 6105–6114, 2019.

Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241, 2015. doi: 10.1007/978-3-319-24574-4_28.